

Tema V – Colas

Tras haber estudiado las pilas y su comportamiento, nos adentramos ahora en el estudio de otra estructura lineal fundamental: la cola (*queue*). Si las pilas se comportan como una pila de platos donde solo podemos operar por la parte superior, las colas se asemejan a una fila de personas esperando su turno: el primero en llegar es el primero en ser atendido. Este principio se conoce como FIFO (*First In, First Out*) y constituye uno de los paradigmas más intuitivos y ampliamente utilizados en la organización de procesos.

Las colas tienen aplicación en innumerables contextos de los sistemas computacionales y del mundo real: desde la gestión de procesos en un sistema operativo hasta la organización de trabajos de impresión, pasando por la atención al cliente en centros de llamadas o la gestión de paquetes en redes de computadoras. La importancia de esta estructura radica en su capacidad para modelar de manera natural procesos donde el orden de llegada determina el orden de atención, garantizando equidad y previsibilidad.

Este capítulo explorará en profundidad el tipo abstracto de datos Cola, analizando sus diferentes implementaciones, complejidades computacionales y aplicaciones prácticas. Veremos cómo, a pesar de su simplicidad conceptual, la implementación eficiente de una cola presenta desafíos interesantes que nos obligarán a aplicar creativamente los conocimientos adquiridos sobre arrays, punteros y gestión de memoria dinámica.

1. El TAD Cola

Un Tipo Abstracto de Datos Cola define una estructura lineal que sigue estrictamente el principio FIFO. Su especificación formal en lenguaje natural sería:

- **Nombre:** Cola
- **Valores posibles:** Vacía o conteniendo una secuencia ordenada de elementos del tipo base, donde solo los elementos en los extremos (primero y último) son directamente accesibles.
- **Operaciones básicas:**
 - `initialize_queue(c)`: Inicializa una cola vacía.
 - `enqueue(c, x)`: Añade el elemento `x` al final de la cola `c`.
 - `dequeue(c)`: Elimina el elemento situado al frente de la cola `c`.
 - `first(c)`: Consulta (sin eliminar) el elemento situado al frente de la cola `c`.
 - `last(c)`: Consulta (sin eliminar) el elemento situado al final de la cola `c`.
 - `is_empty(c)`: Determina si la cola `c` no contiene elementos.
- **Uso de otros tipos:** Depende del tipo base de elementos que contendrá la cola y de las estructuras auxiliares utilizadas en la implementación.

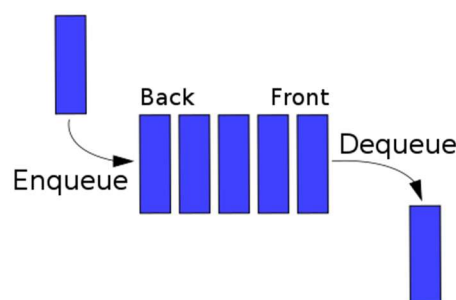


Ilustración 10. Operaciones básicas de un TAD Cola

Algunas operaciones conllevan precondiciones que garantizan su correcto funcionamiento:

- **enqueue** (*c*, *x*) : Añade *x* al final de *c*. *Precondición*: La cola no debe estar llena (solo para implementaciones con capacidad limitada).
- **dequeue** (*c*) : Elimina el elemento del frente de *c*. *Precondición*: La cola no debe estar vacía pues intentar hacer dequeue en una cola vacía es un error (*underflow*).
- **first** (*c*) : Devuelve el elemento del frente sin modificarlo. *Precondición*: La cola no debe estar vacía.
- **last** (*c*) : Devuelve el elemento del final sin modificarlo. *Precondición*: La cola no debe estar vacía.

La nomenclatura de estas operaciones varía según los lenguajes y contextos: enqueue/dequeue son comunes en contextos académicos, mientras que en algunas APIs se usan add/remove o offer/poll.

Además de las operaciones básicas, a menudo resulta útil extender el TAD Cola con funcionalidades adicionales tales como:

- **clear** (*c*) : Vacía completamente la cola.
- **size** (*c*) o **num_elems** (*c*) : Devuelve el número de elementos actualmente en la cola.
- **toString** (*c*) : Genera una representación textual de la cola.
- **copy** (*c1*, *c2*) : Crea una copia independiente de la cola *c1* en *c2*.
- **isFull** (*c*) : Determina si la cola ha alcanzado su capacidad máxima (en implementaciones con límite).

Estas operaciones pueden implementarse eficientemente o, en algunos casos, requerirán recorrer toda la cola, afectando su complejidad temporal.

2. Implementación con arrays

Mientras que la implementación de pilas con arrays es relativamente sencilla (solo necesitamos un índice para la cima), las colas presentan un desafío mayor: necesitamos gestionar dos extremos (frente y final) que se mueven independientemente. Esta dualidad genera varios problemas de diseño, dando lugar a múltiples estrategias de implementación, cada una con sus ventajas e inconvenientes.

A continuación veremos algunas alternativas de implementación.

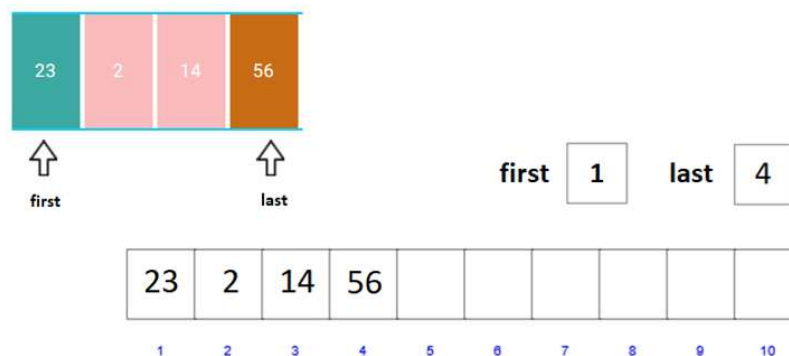


Ilustración 11. Esquema de un TAD cola implementado con arrays

Array lineal con eliminación simple

Mantenemos dos índices: `first` (frente) y `last` (final). La inserción siempre ocurre al final, incrementando `last`. La eliminación simplemente incrementa `first`.

Ejemplo visual:

Estado inicial: `first=1`, `last=0` (cola vacía)

[| | | |] (array de tamaño 5)

Después de `enqueue(10)`: `first=1`, `last=1`

[10| | |]

Después de `enqueue(20)`, `enqueue(30)`: `first=1`, `last=3`

[10|20|30| |]

Después de `dequeue()`: `first=2`, `last=3`

[10|20|30| |] (el 10 sigue en memoria pero ya no es parte de la cola)

Este enfoque tiene la ventaja de que su implementación es extremadamente sencilla, y las operaciones `enqueue` y `dequeue` son $O(1)$. Sin embargo, se desaprovecha espacio pues a medida que hacemos `dequeue`, el espacio al inicio del array queda inutilizable. Aunque el array pueda tener espacio libre al inicio, cuando `last` llega al final, consideramos la cola llena lo que en la práctica significa que los elementos "eliminados" siguen ocupando memoria.

En definitiva, esta implementación es insostenible para usos prolongados, ya que desperdicia progresivamente la capacidad del array.

Array lineal con reorganización

Para evitar el problema del agotamiento ficticio del espacio del modelo anterior, ahora cada vez que eliminemos un elemento, desplazaremos todos los elementos restantes una posición hacia el inicio, manteniendo `first` siempre en la posición 1.

Ejemplo visual:

Estado inicial:

[10|20|30|40|], `first=1`, `last=4`

Después de `dequeue()` y reorganizar:

[20|30|40| |], `first=1`, `last=3`

Esto obviamente complica la implementación del procedimiento de eliminación que no se limita únicamente a mover el índice `last` sino que incorpora reorganización:

```
procedure dequeue(var c: tColaArray);
var i: integer;
begin
  if not empty_queue(c) then begin
    { Desplazar todos los elementos hacia la izquierda }
    for i := 1 to c.last-1 do
      c.array[i] := c.array[i+1];
    c.last := c.last - 1;
  end;
end;
```

Este enfoque sí utiliza eficientemente todo el espacio del array, a diferencia del modelo anterior, dado que `first` siempre es 1 lo que además simplifica algunas operaciones. Sin embargo, `dequeue` es $O(n)$ ya que debemos desplazar $n-1$ elementos tras cada eliminación, lo que resulta un coste inaceptable para colas grandes. Así, esta implementación solo es adecuada para colas muy pequeñas o para colas en las que las eliminaciones son infrecuentes.

Array circular

Otra posibilidad para implementar una cola mediante la utilización como contenedor de elementos de un array, es tratar el array como un “anillo” donde el último elemento está lógicamente enlazado con el primero, usando aritmética modular para calcular las posiciones.

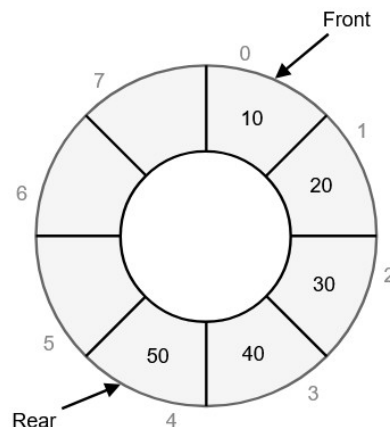


Ilustración 12. Implementación de una Cola mediante array circular

En esta implementación, el cálculo de posiciones debe utilizar aritmética modular, tanto en la inserción como en la eliminación. Así:

```
last := (last mod TAM) + 1; { Para insertar (enqueue) }
first := (first mod TAM) + 1; { Para eliminar (dequeue) }
```

Ejemplo visual (TAM=5):

Estado: `first=1`, `last=3`, `count=2`

[10|20| | |] (10 en posición 1, 20 en posición 2)

Después de `enqueue(30)`: `first=1`, `last=4`, `count=3`

[10|20|30| |]

Después de `dequeue()`: `first=2`, `last=4`, `count=2`

[|20|30| |] (la posición 1 está ahora disponible para reuso)

Después de `enqueue(40)`, `enqueue(50)`, `enqueue(60)`: `first=2`, `last=2`, `count=5`

[60|20|30|40|50] (cola llena, 60 ocupó la posición 5 y "volvió al inicio")

La ventaja de este modelo es que utiliza eficientemente todo el espacio, sin perjudicar el rendimiento ya que las operaciones `enqueue` y `dequeue` son $O(1)$ pues no requiere reorganización. Por otra parte, la implementación es algo más compleja que el modelo lineal, pero, sobre todo, sigue adoleciendo del problema fundamental de todas las implementaciones con array: el tamaño máximo es fijo, con todas las desventajas ya detalladas anteriormente.

Una posible implementación en Pascal sería la siguiente:

```
unit uColaCircular;

interface

    const
        TAM = 100;

    type
        tColaCircular = record
            elementos: array[1..TAM] of integer;
            first: integer; { Índice del primer elemento }
            last: integer;  { Índice donde se insertará el próximo }
            count: integer; { Número actual de elementos }
        end;

    procedure initialize_queue(var c: tColaCircular);
    function is_empty(c: tColaCircular): boolean;
    function is_full(c: tColaCircular): boolean;
    procedure enqueue(var c: tColaCircular; x: integer);
    procedure dequeue(var c: tColaCircular);
    function first(c: tColaCircular): integer;

implementation

    procedure initialize_queue(var c: tColaCircular);
    begin
        c.first := 1;
        c.last := 1;
        c.count := 0;
    end;

    function is_empty(c: tColaCircular): boolean;
    begin
        is_empty := (c.count = 0);
    end;

    function is_full(c: tColaCircular): boolean;
    begin
        is_full := (c.count = TAM);
    end;

    procedure enqueue(var c: tColaCircular; x: integer);
    begin
        if not is_full(c) then begin
            c.elementos[c.last] := x;
            c.last := (c.last mod TAM) + 1;
            c.count := c.count + 1;
        end;
    end;

    procedure dequeue(var c: tColaCircular);
    begin
        if not is_empty(c) then begin
            c.first := (c.first mod TAM) + 1;
            c.count := c.count - 1;
        end;
    end;

    function first(c: tColaCircular): integer;
    begin
        if not is_empty(c) then
            first := c.elementos[c.first];
    end;
```

Comparativa de implementaciones con arrays

Como resumen de lo visto hasta el momento, el siguiente cuadro analiza la complejidad de las operaciones de cada modelo de implementación:

Característica	Eliminación simple	Con reorganización	Array circular
Complejidad enqueue	O(1)	O(1)	O(1)
Complejidad dequeue	O(1)	O(n)	O(1)
Uso de memoria	Ineficiente (desperdicia)	Eficiente	Eficiente
Complejidad implementación	Muy simple	Simple	Moderada
Tamaño máximo	Fijo, mal aprovechado	Fijo, bien aprovechado	Fijo, bien aprovechado
Casos de uso	Solo demostración	Colas muy pequeñas	Producción (tamaño conocido)

La tabla muestra cómo el array circular es la única implementación con arrays que ofrece operaciones O(1) y uso eficiente de memoria, por lo que es la preferida cuando se conoce un tamaño máximo razonable. Si el tamaño máximo no se conoce, ninguna de las implementaciones basadas en un array sería aceptable.

3. Implementación con memoria dinámica

Si bien existen otras posibilidades, la implementación dinámica más sencilla de una cola utiliza una lista enlazada simple de elementos, donde cada nodo contiene el dato a almacenar y un puntero al siguiente nodo. Esta implementación mantiene dos punteros: `first` (al primer nodo) y `last` (al último nodo) y, lógicamente, una cola vacía tiene ambos punteros a `nil`.

Las ventajas de la implementación sobre arrays es que desaparece la limitación de tamaño máximo, lógicamente excepto el límite de la memoria disponible en el sistema sobre el que se ejecute. Tampoco hay desperdicio de espacio ni se necesitan reorganizaciones. Las operaciones `enqueue` y `dequeue` son eficientes, como veremos, pues su complejidad es O(1).

También presenta algunas desventajas, como la necesidad de mayor memoria por elemento (ya que se necesita espacio para el puntero), o que es necesaria una gestión explícita de memoria en lenguajes sin recolector automático de basura (uso de operaciones `new/dispose`).

Pascal

```
unit uColaEnteros;

interface

type
  PNode = ^TNode;
  TNode = record
    info: integer;
    siguiente: PNode;
  end;

  TCola = record
    primero: PNode; { Frente de la cola }
    ultimo: PNode;  { Final de la cola }
  end;

procedure initialize_queue(var c: TCola);
function empty_queue(c: TCola): boolean;
procedure enqueue(var c: TCola; x: integer);
procedure dequeue(var c: TCola);
```

```

function first(c: TCola): integer;
function last(c: TCola): integer;

implementation

procedure initialize_queue(var c: TCola);
begin
    c.primeros := nil;
    c.ultimo := nil;
end;

function empty_queue(c: TCola): boolean;
begin
    empty_queue := (c.primeros = nil);
end;

procedure enqueue(var c: TCola; x: integer);
var
    nuevo: PNode;
begin
    new(nuevo);
    nuevo^.info := x;
    nuevo^.siguiente := nil;
    if empty_queue(c) then c.primeros := nuevo
    else c.ultimo^.siguiente := nuevo;
    c.ultimo := nuevo;
end;

procedure dequeue(var c: TCola);
var
    aux: PNode;
begin
    if not empty_queue(c) then begin
        aux := c.primeros;
        c.primeros := c.primeros^.siguiente;
        if c.primeros = nil then
            c.ultimo := nil;
        dispose(aux);
    end
    else
        writeln('Error: Cola vacía - no se puede hacer dequeue');
    end;
end;

function first(c: TCola): integer;
begin
    if not empty_queue(c) then
        first := c.primeros^.info
    else begin
        writeln('Error: Cola vacía - no se puede consultar first');
        first := 0; { Valor por defecto }
    end;
end;

function last(c: TCola): integer;
begin
    if not empty_queue(c) then
        last := c.ultimo^.info
    else begin
        writeln('Error: Cola vacía - no se puede consultar last');
        last := 0; { Valor por defecto }
    end;
end;

end.

```

Podemos ampliar nuestro TAD Cola con algunas operaciones extra tales como:

```
procedure clear(var c: TCola);
function size(c: TCola): integer;
function toString(c: TCola): string;
```

cuya implementación sería:

```
procedure clear(var c: TCola); { Vacía completamente la cola }
begin
  while not empty_queue(c) do
    dequeue(c);
end;

function size(c: TCola): integer; { Cuenta el número de elementos (O(n)) }
var
  actual: PNode;
  contador: integer;
begin
  actual := c.primer;
  contador := 0;
  while actual <> nil do begin
    contador := contador + 1;
    actual := actual.sig;
  end;
  size := contador;
end;

{ Versión O(1) de size: requiere modificar la estructura TCola
  para incluir un campo "cantidad" que se actualizaría en enqueue/dequeue }

function toString(c: TCola): string; { Representación textual de la cola }
var
  actual: PNode;
  resultado: string;
begin
  resultado := '[';
  actual := c.primer;
  while actual <> nil do begin
    resultado := resultado + IntToStr(actual.info);
    actual := actual.sig;
    if actual <> nil then
      resultado := resultado + ', ';
  end;
  resultado := resultado + ']';
  toString := resultado;
end;
```

4. Aplicaciones prácticas de las colas

Una de las aplicaciones más comunes de las colas es la planificación de procesos en los sistemas operativos, donde distintos algoritmos, como FIFO Simple (First-Come, First-Served) y Round Robin, organizan la ejecución de tareas, asegurando equidad y eficiencia en el uso de la CPU. De manera similar, los sistemas de impresión emplean colas para gestionar trabajos mediante spooling, encolando cada documento enviado por múltiples usuarios y procesándolos en orden FIFO, lo que previene conflictos y garantiza un acceso ordenado al recurso compartido.

En el ámbito de las redes, los routers y switches utilizan colas para administrar el tráfico de paquetes, encolando los que llegan por un puerto para ser enviados por otro y aplicando políticas como FIFO para priorizar la transmisión durante congestiones. Los centros de atención al cliente, a través de sistemas de distribución automática de llamadas, también se basan en colas para

gestionar las llamadas entrantes, asignar agentes a las llamadas más antiguas, informar a los usuarios sobre su tiempo de espera e implementar prioridades para clientes especiales.

Otro uso muy común es la simulación de eventos discretos (simulaciones de tráfico de vehículos, líneas de producción o servicios de atención), donde las colas permiten modelar eficazmente situaciones como el comportamiento de los vehículos en un semáforo, los clientes en las cajas de un supermercado, los aviones en espera de despegue o aterrizaje, o los paquetes que aguardan en líneas de ensamblaje, demostrando así la versatilidad y utilidad de esta estructura en la representación de sistemas donde el orden de llegada determina el orden de atención.

5. Complejidad computacional de las operaciones

Para ambas implementaciones principales (array circular y lista enlazada), las complejidades son:

Operación	Array circular	Lista enlazada	Comentario
<code>initialize_queue</code>	$O(1)$	$O(1)$	
<code>empty_queue</code>	$O(1)$	$O(1)$	
<code>enqueue</code>	$O(1)$	$O(1)$	Amortizado $O(1)$ en lista
<code>dequeue</code>	$O(1)$	$O(1)$	
<code>first</code>	$O(1)$	$O(1)$	
<code>last</code>	$O(1)$	$O(1)$	Requiere puntero al último
<code>size</code>	$O(1)$	$O(n)$ o $O(1)^*$	*Con contador adicional
<code>clear</code>	$O(1)$	$O(n)$	Liberar todos los nodos
<code>toString</code>	$O(n)$	$O(n)$	

A la vista del cuadro anterior, hacemos un par de observaciones importantes relativas a la operación extra `clear`. En el array circular `clear` es $O(1)$ porque solo necesita resetear índices, no borrar elementos. En la lista enlazada, sin embargo, `clear` es $O(n)$ porque debe liberar cada nodo individualmente. Resaltar que la implementación con contador para `size` en lista enlazada es un buen ejemplo de optimización espacio-tiempo.

En cuanto a los factores a considerar a la hora de elegir una u otra implementación, debemos destacar los siguientes:

- **Tamaño:** Si el tamaño máximo es conocido y razonable elegimos un array circular, si es impredecible o muy variable la implementación idónea es la lista enlazada.
- **Frecuencia de operaciones:** Para uso intensivo elegiremos un array circular, pero si vamos a hacer un uso esporádico entonces mejor la lista enlazada (flexibilidad).
- **Requisitos de memoria:** Será muy raro pero si la memoria es muy limitada puede ser mejor el array circular (ya que el overhead es fijo), mientras que si la memoria es abundante optaremos por la lista enlazada, que puede crecer.
- **Lenguaje y plataforma:** En lenguajes con recolector de basura (Java, Python) es preferible la implementación con lista enlazada, más simple, pero en lenguajes con gestión manual (C, Pascal) es mejor un array circular, pues evita el engorro de la gestión de memoria.

En resumen: La elección depende del contexto específico: para tamaños conocidos y predecibles, el array circular ofrece mejor rendimiento; para tamaños variables o desconocidos, la lista enlazada proporciona flexibilidad. Ambas garantizan operaciones $O(1)$ para las funcionalidades básicas, cumpliendo con los requisitos de eficiencia de la mayoría de las aplicaciones.

En el próximo capítulo, exploraremos estructuras más complejas como las listas enlazadas, que generalizan tanto pilas como colas, permitiendo operaciones de inserción y eliminación en cualquier posición. Sin embargo, la comprensión sólida de pilas y colas será fundamental para abordar esas estructuras más generales, ya que muchas de sus operaciones se implementan utilizando pilas y colas como componentes auxiliares.